

KARNATAKA UNIVERSITY MATHEMATICS TEACHERS
ASSOCIATION (KUMTA)
B.Sc. SEM – IV/V/VI (REVISED NEP)-2025-26
MATHEMATICS SEC LABORATORY MANUAL
Course Title: Programming with GNU Octave
Prepared by: Dr. Shivaprasad Nayaka, JSS College Dharwad.

GNU OCTAVE: INSTALLATION, FEATURES AND BASIC USAGE

The main goal of this lab is to understand the installation process, working environment and fundamental features of GNU Octave. You will learn how to write, save and execute programs using the GNU Octave GUI, which provides a user-friendly interface for performing mathematical, scientific and engineering computations. By the end of this lab, you will be able to perform basic arithmetic, matrix operations, calculus and symbolic computations, gaining confidence in using Octave for both academic and research purposes.

DOWNLOAD AND INSTALLATION OF GNU OCTAVE

GNU Octave is free, open-source software widely used for numerical computation. It can run on Windows, Linux, and macOS, making it highly versatile.

Steps to Install on Windows:

- Open your web browser and go to the official GNU Octave website.
- Click on the download option and select the Windows version.
- Download the installer file (usually ending with .exe).
- Double-click the downloaded file to start the installation. The setup wizard will appear and guide you through each step.
- Accept the license agreement and click Next.
- Stick with the default installation settings, unless you prefer a different directory.
- Click Finish when the installation is complete.

After installation, you can launch Octave from the Start Menu by selecting GNU Octave (GUI).

GNU OCTAVE INTERFACES

Once installed, Octave can be run in two modes:

1. Graphical User Interface (GUI):

- Provides a user friendly environment with windows for commands, code editing and variable display.
- Includes the Command Window, Editor Window and Workspace, making it easier to write, run and debug programs.
- This mode is preferred for learning, laboratory work and academic purposes.

2. Command Line Interface (CUI):

- Operates purely with text based commands without graphical support.
- Although functional, it's less intuitive than the GUI for beginners.

In the laboratory exercises, we focus on the GUI mode, writing programs in the Editor Window with `.m` extension files and executing them to see results in the Command Window.

INTRODUCTION TO GNU OCTAVE

GNU Octave is high-level numerical computation software designed for solving complex mathematical, scientific, and engineering problems. It is particularly powerful in areas such as: Linear algebra, Calculus, Differential equations, Integral transforms, Group theory and Numerical analysis.

A key advantage of Octave is its compatibility with MATLAB. This means MATLAB programs can run with little or no modification, making Octave a popular free alternative for educational and research purposes.

Its open-source nature, combined with a wide range of mathematical tools, has made it an invaluable software for students, teachers and researchers.

FEATURES OF GNU OCTAVE

GNU Octave offers several features that make it highly suitable for both learning and research:

- Efficient handling of matrix and vector operations.
- Numerous built-in functions for linear algebra and numerical computations.
- Support for symbolic mathematics through the Symbolic Package.
- Capability to perform differentiation and integration.
- Tools for Laplace and Fourier transforms.
- Support for algebraic structures such as groups and Cayley tables.
- Script based programming using .m files.
- Cross-platform compatibility (Windows, Linux, macOS).

GNU OCTAVE ENVIRONMENT

The Octave GUI provides a structured environment to simplify programming and computation:

1. Command Window:

- The main area for executing commands line by line.
- Displays outputs immediately, making it ideal for testing and quick calculations.

2. Editor Window:

- Used to write, edit and save programs.
- Programs are saved with the .m extension and can be executed multiple times.

3. Workspace:

- Shows all defined variables and their current values.
- Helps track calculations and avoid errors in large programs.

USING THE COMMAND WINDOW

The Command Window allows immediate execution of commands without saving a program.

Some examples include:

1. Basic arithmetic:

```
>> 5 + 3
```

```
ans = 8
```

```
>> 10 / 2
```

```
ans = 5
```

2. Matrix operations:

```
>> A = [1 2 3; 4 5 6; 7 8 9];
```

```
>> B = [9 8 7; 6 5 4; 3 2 1];
```

```
>> A + B
```

```
ans =
```

```
10 10 10
```

```
10 10 10
```

```
10 10 10
```

```
>> A * B
```

```
ans =
```

```
30 24 18
```

```
84 69 54
```

```
138 114 90
```

3. Calculus using the Symbolic Package:

```
>> pkg load symbolic
```

```
>> syms x
```

```
>> diff(sin(x), x)
```

```
ans = cos(x)
```

```
>> int(x^2, x)
```

```
ans = x^3/3
```

4. Solving equations:

```
>> solve(x^2 - 5*x + 6 == 0, x)
ans =
    2
    3
```

5. Symbolic variable example:

```
>> syms t a
>> f = exp(a*t)
f = exp(a*t)

>> diff(f, t)
ans = a*exp(a*t)
```

SYMBOLIC PACKAGE IN GNU OCTAVE

By default, Octave performs numerical computations. To work with symbolic mathematics, the Symbolic Package must be installed and loaded:

```
pkg load symbolic;
```

Once loaded, this package enables functions for:

- Defining symbolic variables (`syms`)
- Differentiation (`diff`)
- Integration (`int`)
- Laplace transforms (`laplace`)
- Fourier transforms (`fourier`)

Symbolic computation is particularly useful when dealing with algebraic expressions, solving differential equations or performing complex calculus operations.

SYMBOLIC VARIABLES

Symbolic variables allow Octave to handle variables algebraically rather than numerically.

For example:

```
syms x a;
```

Here, x and a are symbolic variables which can then be used in differentiation, integration and other symbolic computations. Symbolic variables are essential when working on problems that require exact mathematical expressions rather than approximate numerical results.

PROCEDURE TO EXECUTE A PROGRAM IN GNU OCTAVE

Executing a program in Octave involves the following steps:

- Open GNU Octave (GUI).
- Launch the Editor Window.
- Type the program or script you want to execute.
- Save the file with a `.m` extension (e.g., `matrix_addition.m`).
- Click the Run button or press **F5** to execute the program.
- Observe the output in the Command Window.

LIST OF EXPERIMENTS

1. Program to compute addition and subtraction of matrices.
2. Program to compute rank of matrix and row reduced echelon form.
3. Program to find the angle between the radius vector and tangent.
4. Program to find the n th derivative of e^{ax} and trigonometric functions.
5. Program to construct Cayley table and test abelian for given finite set.
6. Programs to verify Lagrange's theorem (groups) with suitable example.
7. Program to evaluate the double and triple integrals with constant limits.
8. Program to verify the exactness of a differential equation.
9. Program to find the Complementary Function and Particular Integral of linear differential equations with constant coefficients.
10. Program to verify D'Alembert's and Raabe's tests with suitable examples.

11. Program to find the Laplace transforms of some standard and periodic functions.

12. Program to find sine and cosine Fourier transforms.

Program 1: Program to compute addition and subtraction of matrices

```
clc;clear;
A = [1 2 3; 2 4 6; 4 5 6];
B = [6 5 4; 3 5 7; 3 2 1];
disp('The first matrix is');
disp(A);
disp('The second matrix is');
disp(B);
[m, n] = size(A);
for i = 1:m
    for j = 1:n
        C(i, j) = A(i, j) + B(i, j);
        D(i, j) = A(i, j) - B(i, j);
    end
end
disp('Addition of matrices is');
disp(C);
disp('Subtraction of matrices is');
disp(D);
```

Output:

The first matrix is

1 2 3

2 4 6

4 5 6

The second matrix is

6 5 4

3 5 7

3 2 1

Addition of matrices is

7 7 7

5 9 13

7 7 7

Subtraction of matrices is

-5 -3 -1

-1 -1 -1

1 3 5

Program 2: Program to compute rank of matrix and row reduced echelon form.

```
clc;clear;
A = [1 2 3; 2 4 6; 1 1 1];
disp('Given matrix is');
disp(A);
R = rref(A);
disp('Row Reduced Echelon Form of a given matrix');
disp(R);
rank_A = rank(A);
disp('Rank of the matrix A:');
disp(rank_A);
```

Output:

Given matrix is

1 2 3

2 4 6

1 1 1

Row Reduced Echelon Form of a matrix is

1 0 -1

0 1 2

0 0 0

Rank of the given matrix is

2

Program 3: Program to find the angle between the radius vector and tangent

```
clc;clear;
a = 1;
theta = pi/4;
r = a *(1-cos(theta));
dr = a*(sin(theta));
tan_psi = r / dr;
psi_rad = atan(tan_psi);
psi_deg = psi_rad * (180/pi);
disp('Angle between radius vector and tangent (in radian):');
disp(psi_rad);
disp('Angle in degree:');
disp(psi_deg);
```

Output:

Angle between radius vector and tangent (in radian):

0.3927

Angle in degree:

22.500

Program 4: Program to find the nth derivative of e^{ax} and trigonometric functions

```
clc;clear;

pkg load symbolic;

syms x a b;

n = 4;

f_exp = exp(a*x);

f_sin = sin(a*x + b);

f_cos = cos(a*x + b);

nth_exp = diff(f_exp, x, n);

nth_sin = diff(f_sin, x, n);

nth_cos = diff(f_cos, x, n);

disp('nth derivative of  $e^{ax}$ :');

disp(nth_exp);

disp('nth derivative of  $\sin(ax + b)$ :');

disp(nth_sin);

disp('nth derivative of  $\cos(ax + b)$ :');

disp(nth_cos);
```

Output:

nth derivative of e^{ax} :

$$a^4 * e^{a*x}$$

nth derivative of $\sin(ax + b)$:

$$a^4 * \sin(a*x + b)$$

nth derivative of $\cos(ax + b)$:

$$a^4 * \cos(a*x + b)$$

Program 5: Program to construct Cayley table and test abelian for given finite set

```
clc;clear;
G = [0 1 2];
n = length(G);
for i = 1:n
    for j = 1:n
        C(i, j) = mod(G(i) + G(j), 3);
    end
end
disp('Elements of the group:');
disp(G);
disp('Cayley Table:');
disp(C);
flag = 1;
for i = 1:n
    for j = 1:n
        if C(i, j) ~= C(j, i)
            flag = 0;
            break;
        end
    end
end
if flag == 1
    disp('The given group is ABELIAN');
else
    disp('The given group is NON-ABELIAN');
end
```

Output:

Elements of the group:

0 1 2

Cayley Table:

0 1 2

1 2 0

2 0 1

The given group is ABELIAN

Program 6: Programs to verify Lagrange's theorem (groups) with suitable example

```
clc;clear;
G = [0 1 2 3 4 5];
H = [0 2 4];
order_G = length(G);
order_H = length(H);
disp('Group G = ');
disp(G);
disp(['Order of group G = ', num2str(order_G)]);
disp('Subgroup H = ');
disp(H);
disp(['Order of subgroup H = ', num2str(order_H)]);
if mod(order_G, order_H) == 0
    disp('Lagrange's Theorem is VERIFIED');
else
    disp('Lagrange's Theorem is NOT VERIFIED');
```

Output:

Group G =

0 1 2 3 4 5

Order of group G = 6

Subgroup H =

0 2 4

Order of subgroup H = 3

Lagrange's Theorem is VERIFIED

Program 7: Program to evaluate the double and triple integrals with constant limits

```
clc;clear;
pkg load symbolic;
syms x y z;
f_2 = x * y;
double_integral = int(int(f_2, x, 0, 1), y, 0, 2);
disp('Double integral of f(x,y) = xy over [0,1]x[0,2]:');
disp(double_integral);
f_3 = x * y * z;
triple_integral = int(int(int(f_3, x, 0, 1), y, 0, 1), z, 0, 1);
disp('Triple integral of f(x,y,z) = xyz over [0,1]^3:');
disp(triple_integral);
```

Output:

Double integral of f(x,y) = xy over [0,1]x[0,2]:

1

Triple integral of f(x,y,z) = xyz over [0,1]^3:

1/8

Program 8: Program to verify the exactness of a differential equation

$$(2xy + 3)dx + (x^2 + 4y^3)dy = 0$$

```
clc;clear;
pkg load symbolic;
syms x y;
M = 2*x*y + 3;
```

```

N = x^2 + 4*y^3;
dM_dy = diff(M, y);
dN_dx = diff(N, x);
disp('Partial derivative of M w.r.t y:');
disp(dM_dy);
disp('Partial derivative of N w.r.t x:');
disp(dN_dx);
if simplify(dM_dy - dN_dx) == 0
    disp('The differential equation is EXACT');
else
    disp('The differential equation is NOT EXACT');
end

```

Output:

Partial derivative of M w.r.t y:

$2*x$

Partial derivative of N w.r.t x:

$2*x$

The differential equation is EXACT

Program 9: Program to find the Complementary Function and Particular Integral of linear differential equations with constant coefficients

```

clc;clear;
pkg load symbolic;
syms y(x) C1 C2 x;
Dy = diff(y, x);
D2y = diff(y, x, 2);
ode_h = D2y - y == 0;
CF = dsolve(ode_h);
syms A

```

```

y_p = A*exp(2*x);
lhs = diff(y_p, x, 2) - y_p;
sol_A = solve(lhs == exp(2*x), A);
PI = simplify(subs(y_p, A, sol_A));
sol = CF + PI;
disp('Complementary Function (CF):');
disp(char(CF));
disp('Particular Integral (PI):');
disp(char(PI));
disp('General Solution y(x) = CF + PI:');
disp(char(sol));

```

Output:

```

Complementary Function (CF):
C1*exp(-x) + C2*exp(x)
Particular Integral (PI):
exp(2*x)/3
General Solution y(x) = CF + PI:
C1*exp(-x) + C2*exp(x) + exp(2*x)/3

```

Program 10: Program to verify D'Alembert's and Raabe's tests with suitable examples

```

clc;clear;
pkg load symbolic;
syms n;
a_n = factorial(n)/n^n;
a_n1 = factorial(n+1)/(n+1)^(n+1);
ratio = simplify(a_n1 / a_n);
L = limit(ratio, n, inf);
disp(' D"Alembert"s Ratio Test');

```

```

disp('Series: a_n = n!/n^n');
disp(['a_(n+1)/a_n = ', char(ratio)]);
disp(['Limit L = ', char(L)]);
if double(L) < 1
    disp('Series converges by D'Alembert's test');
elseif double(L) > 1
    disp('Series diverges by D'Alembert's test');
else
    disp('D'Alembert's test fail');
end
a_n = 1/n^2;
a_n1 = 1/(n+1)^2;
L = limit(n*(a_n / a_n1 - 1), n, inf);
disp(' ');
disp(' Raabe's Test');
disp('Series: a_n = 1/n^2');
disp(['L = ', char(L)]);
if double(L) > 1
    disp('Series converges by Raabe's test');
elseif double(L) < 1
    disp('Series diverges by Raabe's test');
else
    disp('Raabe's test fail');
end

```

Output:

D'Alembert's Ratio Test

Series: $a_n = n!/n^n$

$$a_{n+1}/a_n = n^{**n}/(n + 1)^{**n}$$

$$\text{Limit } L = \exp(-1)$$

Series converges by D'Alembert's test

Raabe's Test

$$\text{Series: } a_n = 1/n^2$$

$$L = 2$$

Series converges by Raabe's test

Program 11: Program to find the Laplace transforms of some standard and periodic functions

```
clc;clear;
pkg load symbolic;
syms t s a m n;
% 1. f(t) = exp(a*t)
f1 = exp(a*t);
F1 = laplace(f1, t, s);
disp('Laplace Transform of exp(a*t):');
disp(F1);
% 2. f(t) = sin(m*t)
f2 = sin(m*t);
F2 = laplace(f2, t, s);
disp('Laplace Transform of sin(a*t):');
disp(F2);
% 3. f(t) = cos(m*t)
f3 = cos(m*t);
F3 = laplace(f3, t, s);
```

```

disp('Laplace Transform of cos(a*t):');
disp(F3);
% 4. f(t) = exp(a*t)*cos(m*t)
f1 = exp(a*t)*cos(m*t);
F1 = laplace(f1, t, s);
disp('Laplace Transform of e^(a*t)*cos(m*t):');
disp(F1);
% 5. f(t) = exp(a*t)*sin(m*t)
f2 = exp(a*t)*sin(m*t);
F2 = laplace(f2, t, s);
disp('Laplace Transform of e^(a*t)*sin(m*t):');
disp(F2);
% 6. f(t) = t^n
f3 = t^n;
F3 = laplace(f3, t, s);
disp('Laplace Transform of t^n:');
disp(F3);

```

Output:

Laplace Transform of $\exp(a*t)$:

$$\frac{1}{-a + s}$$

Laplace Transform of $\sin(a*t)$:

$$\frac{m}{m^2 + s^2}$$

Laplace Transform of $\cos(a*t)$:

$$\frac{s}{m^2 + s^2}$$

Laplace Transform of $e^{at}\cos(mt)$:

$$\frac{-a + s}{m^2 + (-a + s)^2}$$

Laplace Transform of $e^{at}\sin(mt)$:

$$\frac{m}{m^2 + (-a + s)^2}$$

Laplace Transform of t^n :

$$\frac{s^{-n} \Gamma(n + 1)}{s}$$

Program 12: Program to find sine and cosine Fourier transforms

```
clc;clear;
pkg load symbolic;
syms t s a positive;
f = exp(-a*t);
FST = sqrt(2/sym('pi')) * s / (a^2 + s^2);
disp('Fourier Sine Transform of f(t) = exp(-a*t):');
disp(FST);
FCT = sqrt(2/sym('pi')) * a / (a^2 + s^2);
disp('Fourier Cosine Transform of f(t) = exp(-a*t):');
disp(FCT);
```

Output:

Fourier Sine Transform of $f(t) = \exp(-at)$:

$$\sqrt{2} * s$$

$$\sqrt{\pi} (a^2 + s^2)$$

Fourier Cosine Transform of $f(t) = \exp(-a \cdot t)$:

$$\sqrt{2} \cdot a$$

$$\sqrt{\pi} (a^2 + s^2)$$

KUMTA